

وثيقة التصميم الفني وبنية البرمجيات

منصة هندسة الكشف والعمليات الأمنية الموحدة (Unified SOC Platform)

وثيقة مرجعية شاملة تغطي الهيكل البنائي، الروابط البرمجية (APIs)، تصميم محرك الاختبار المعزول وآلية عمل هندسة الكشف لتبسيط التطوير وتكامل الأنظمة، (Sandbox).

مشروع التخرج: منصة إدارة وهندسة كشف التهديدات السيبرانية
طبيعة المستند: دليل المطور ومواصفات الباك اند (Backend Blueprint)
تاريخ الإصدار: مايو 2026
الحالة: جاهز للتنفيذ الفوري

1. نظرة عامة على النظام ومكوناته الفنية

تُمثل هذه المنصة نظاماً متكاملًا لإدارة هندسة الكشف (Detection Engineering) وعمليات مراقبة الأمن (SOC). تهدف المنصة إلى أتمتة دورة حياة قواعد الاكتشاف بالكامل؛ بدءاً من كتابة القواعد بلغة موحدة، واختبارها داخل بيئة معزولة آمنة، وتحويلها إلى لغات تشغيلية لمنصات السيام (SIEM) المختلفة، ووصولاً إلى تحسين الكفاءة عبر تقليل التنبيهات الكاذبة (False Positives).

1.1 المكونات الوظيفية الرئيسية للمنصة

- **لوحة التحكم العملياتية (Security Operations Dashboard):** تعرض الحالة الصحية للنظام، وعدد القواعد النشطة، والتنبيهات الحرجة، ومؤشرات التغطية الأمنية طبقاً لإطار MITRE ATT&CK.
- **محرر القواعد الذكي ومحول لغات الـ SIEM (Rule Builder & Sigma Converter):** واجهة تفاعلية تمكن المحلل من كتابة القواعد بصيغة Sigma YAML، مع التحويل الآلي الفوري لـ Elastic KQL, Splunk SPL, Azure Sentinel, Chronicle).
- **مختبر فحص قواعد الاكتشاف (Simulation & Testing Lab):** بيئة معزولة (Sandbox) لتنفيذ نصوص برمجية خبيثة هجومية (PowerShell, Python, Bash) للتأكد من فاعلية القاعدة المصممة قبل نشرها في البيئة الفعلية.
- **مكتبة سيناريوهات الهجمات المتقدمة (Attack Scenarios Library):** أرشيف يحاكي مجموعات التهديد المستمر (APT Groups) مثل Lazarus و APT29، لتوليد سجلات (Logs) حقيقية تُختبر القواعد بناءً عليها.
- **نظام تصفية وتقليل التنبيهات الكاذبة (False Positive Tuning):** لوحة تحليلات تستخدم خوارزميات ذكية لتحديد القواعد المسببة للإزعاج (Noisy Rules) وحساب الساعات المهدرة وتكلفة التحقيق، مع تقديم مقترحات لضبط المعايير.

2. المعمارية التقنية ومواصفات الباك اند (Backend Specification)

يعتمد الباك اند على بنية معيارية (Modular / Microservices-ready) تضمن الفصل الكامل بين الخدمات الإدارية (تنظيم القواعد والمستخدمين) والخدمات التشغيلية الحرجة (تحويل السيجما وتشغيل المعمل المعزول).

2.1 الهيكل البنائي العام للخدمات (Service Architecture)

يتكون الباك اند من أربع خدمات أساسية تتبادل البيانات عبر بروتوكولات RESTful APIs و Message Queues في حالة العمليات طويلة الأمد (مثل محاكاة هجمات معقدة):

1. **API Gateway & Core Service:** مسؤولة عن إدارة المصادقة، والـ CRUD للقواعد، وحفظ بيانات لوحة التحكم.
2. **Sigma Engine Service:** خدمة مخصصة مبنية على بايثون للتعامل مع مكتبات pySigma لمعالجة وفحص ملفات YAML وتحويلها.

3. **Isolated Sandbox Execution Service:** الخدمة المسؤولة عن إدارة الحاويات (Docker). واختبار البرمجيات الهجومية بحذر وأمان (Containers).
4. **Analytics & Tuning Module:** محرك احتساب كفاءة القواعد وحساب التكلفة وساعات العمل الضائعة بناءً على سجلات الـ SIEM المربوطة.

2.2 مواصفات روابط الواجهة البرمجية (RESTful API Endpoints)

يجب على مهندس الباك اند الالتزام ببناء وهيكلة الروابط التالية بدقة مع إرجاع استجابات بصيغة JSON القياسية:

2.2.1 إدارة وبناء القواعد (Rule Builder & Converter)

المدخلات الرئيسية (Payload)	الوصف الفني ووظيفة الرابط	الطريقة (Method)	الرابط (Endpoint)
{ "yaml_content": "..."}	التحقق من سلامة بناء ملف وصحته Sigma YAML النحوية.	POST	/api/v1/rules/validate
{ "yaml_content": "...", "targets": ["kql", "spl"]}	تحويل قاعدة سيجما إلى الصيغ المستهدفة (KQL, SPL, إلخ).	POST	/api/v1/rules/convert
بيانات القاعدة + التكتيكات + المعرف الرقمي.	حفظ القاعدة الجديدة في قاعدة البيانات وتحديث الربط بـ MITRE.	POST	/api/v1/rules/save
severity, Query params: status	جلب جميع قواعد الاكتشاف المضافة مع الفلترة حسب الخطورة.	GET	/api/v1/rules/list

2.2.2 بيئة المعمل واختبار السيناريوهات (Testing Lab & Sandbox)

الرابط (Endpoint)	الطريقة (Method)	الوصف الفني وطيفة الرابط	المدخلات الرئيسية (Payload)
/api/v1/lab/execute-script	POST	حقن نص برمجي مخصص داخل الحاوية المعزولة وتشغيله.	{ "language": "powershell", "script": "...", "timeout": 30 }
/api/v1/lab/execute-scenario	POST	تشغيل سيناريو APT جاهز من المكتبة ومراقبة النتيجة.	{ "scenario_id": "APT29_Empire_C2" }
/api/v1/scenarios/list	GET	استعراض مكتبة السيناريوهات المتاحة والتفاصيل الخاصة بكل منها.	لا يوجد (تصفح عام للمكتبة)

2.2.3 تصفية التنبيهات الكاذبة (False Positive Tuning)

الرابط (Endpoint)	الطريقة (Method)	الوصف الفني وطيفة الرابط	المدخلات الرئيسية (Payload)
/api/v1/tuning/noisy-rules	GET	جلب القواعد الأعلى تسجيلاً للتنبيهات الكاذبة ومعدلات هدر الوقت.	لا يوجد
/api/v1/tuning/suggestions	GET	استدعاء الاقتراحات الذكية المعدة لكل قاعدة لتقليص الضوضاء.	rule_idQuery param:
/api/v1/tuning/apply	POST	تطبيق الفلتر أو الاستثناء البرمجي المقترح على قاعدة البيانات.	{ "rule_id": 12, "exclusion_query": "..."}

2.3 آلية عمل وتصميم البيئة المعزولة (Sandbox Deep-Dive)

تعتبر خدمة **Sandbox** أدق الأجزاء التقنية في الباك اند. يجب على مهندس البرمجيات عدم تنفيذ الـ Scripts المرفوعة مباشرة على السيرفر المستضيف منعا للاختراق الكلي للنظام. الآلية المعتمدة تعتمد على استدعاء **Docker** من داخل كود الباك اند وفق التسلسل التالي:

تسلسل المعالجة الآمن للهجمات المحاكاتية:

1. يتلقى الباك اند النص البرمجي (مثلاً: كود PowerShell خبيث يقوم بجمع صلاحيات ومحاكاة Ransomware).
2. يقوم الباك اند بإنشاء حاوية Docker مؤقتة بشكل ديناميكي بناءً على بيئة التشغيل المطلوبة (مثال: حاوية وندوز مصغرة أو حاوية لينكس مجهزة بـ PowerShell Core).
3. تطبيق قيود الأمان المشددة على الحاوية:
 - تحديد حجم الذاكرة العشوائية القصوى بـ 512MB فقط لضمان عدم استهلاك موارد السيرفر (Memory Capping).
 - تفعيل العزل الشبكي الكامل الكلي (Network Isolation (Network Disabled) لضمان عدم تواصل السكريبت مع سيرفرات اختراق خارجية (No Command & Control Callback).
 - تحديد سقف زمني صارم للتنفيذ لا يتجاوز 30 ثانية لمنع السكريبتات اللانهائية (Timeout Enforcement) التي تسبب تعليق النظام (Denial of Service).
 - جعل نظام الملفات للقراءة فقط Read-only File System باستثناء مجلد مؤقت tmp/.
4. يتم كتابة السكريبت داخل الحاوية وتشغيله، ومن ثم التقاط مخرجات النظام (Stdout / Stderr) وسجلات التنفيذ وسحبها للباك اند فوراً.
5. بمجرد انتهاء الـ 30 ثانية أو انتهاء السكريبت، يتم تدمير الحاوية كلياً بشكل آمن Container Destruction Cleanup & لضمان عدم ترك أي أثر خبيث.

2.4 معمارية قاعدة البيانات والبيانات الوصفية لـ Sigma YAML

يجب تصميم جداول قاعدة البيانات لتستوعب البيانات المهيكلية لملفات Sigma. الجدول الرئيسي للقواعد يجب أن يحتوي على الحقول الكافية لمطابقة الهيكل الافتراضي لـ Sigma كالتالي:

```
Table DetectionRules {
  id: integer [pk, increment]
  title: varchar(255)
  sigma_id: uuid
  status: varchar(50) // experimental, stable, deprecated
  description: text
  logsource_category: varchar(100) // e.g., process_creation
  logsource_product: varchar(100) // e.g., windows
  detection_logic: text // The raw YAML text or parsed JSON condition
  mitre_techniques: text[] // Array of techniques like T1055, T1047
  severity: varchar(50) // low, medium, high, critical
  created_at: timestamp
}
```

3. خطة العمل وتوزيع المهام البرمجية (Backend Tasks Roadmap)

تم تقسيم بناء الباك اند إلى 5 مراحل متتالية (Sprints)، تضمن بناء أساس صلب وتساعد الوظائف تدريجياً لضمان سلامة التكامل وخروج الأداة بأفضل جودة برمجية ممكنة.

المرحلة الأولى: البنية الأساسية وإعداد قاعدة البيانات (Sprint 1)

رمز المهمة	عنوان المهمة البرمجية للمطور	المدخلات والمخرجات المتوقعة (Deliverables)	الزمن المتوقع
BE-101	تهيئة المشروع وإعداد معمارية المجلدات	بناء الهيكل الأساسي للمشروع، إعداد البيئات الافتراضية، وملفات الإعدادات (Environment Variables).	يوم واحد
BE-102	تصميم وهجرة قاعدة البيانات (DB Schema Migration)	إنشاء جداول المستخدمين، القواعد (Rules)، وجدول سجلات التنبيهات مع تفعيل العلاقات بين الجداول.	يومان
BE-103	بناء الـ APIs الأساسية للوحة التحكم (Dashboard Stats)	تطوير روابط إرجاع الإحصائيات العامة (عدد التنبيهات، القواعد، حالة الخدمات الفرعية الحيوية).	يومان

المرحلة الثانية: محرك معالجة وتحويل القواعد (Sprint 2)

رمز المهمة	عنوان المهمة البرمجية للمطور	المدخلات والمخرجات المتوقعة (Deliverables)	الزمن المتوقع
BE-201	دمج مكتبة فحص بناء الـ YAML (Sigma Validator)	رابط يستقبل نصوص YAML ويتحقق من مطابقتها لمعايير Sigma القياسية وإرجاع الأخطاء وسطر الخطأ بدقة.	يومان
BE-202	بناء محرك التحويل البرمجي (Sigma Backend Transpiler)	استدعاء أدوات التحويل البرمجية لتحويل الكود إلى Azure Sentinel و Splunk SPL و Elastic KQL بنجاح.	3 أيام
BE-203	تطوير الـ CRUD الخاص بـ Rule Builder	روابط الحفظ، التعديل، الأرشفة، والبحث المتقدم في القواعد وتصنيفها بناء على تقنيات MITRE.	يومان

المرحلة الثالثة: البيئة المعزولة ومحاكاة الهجمات (Sprint 3)

رمز المهمة	عنوان المهمة البرمجية للمطور	المدخلات والمخرجات المتوقعة (Deliverables)	الزمن المتوقع
BE-301	التكامل مع Docker SDK وإدارة الحاويات آلياً	كود برمجي قادر على إنشاء حاوية Docker مخصصة فوراً عند الطلب وتطبيق قيود الموارد والأمان عليها.	3 أيام
BE-302	بناء موديول معالجة لغات السكريبتات (Runner Module)	تطوير الـ Execution Engine الذي يدعم تشغيل كود بأمان تام Bash وكود Python وكود PowerShell والتقاط المخرجات.	يومان
BE-303	بناء جدول سيناريوهات مجموعات التهديد (APT Library Database)	تطوير رابط لجلب السيناريوهات العشرة الجاهزة (مثل التجريبية Logs مع سجلات الـ APT28 و Lazarus المدمجة معها).	يومان

المرحلة الرابعة: موديول التحليل وتقليل التنبيهات الكاذبة (Sprint 4)

رمز المهمة	عنوان المهمة البرمجية للمطور	المدخلات والمخرجات المتوقعة (Deliverables)	الزمن المتوقع
BE-401	بناء محرك حساب تكلفة وهدر التنبيهات الكاذبة	كود رياضي يحسب ساعات العمل المستهلكة شهرياً والتكلفة المالية الضائعة بناء على متوسط وقت التحقيق المدخل لكل قاعدة.	يومان
BE-402	تطوير نظام توليد مقترحات الضبط الآلي (AI Tuning Engine)	نظام يحلل القواعد المزعجة (مثل تلك ذات معدل خطأ يفوق 80%) و يولد شروط استثناء تقترح استبعاد مستخدمي شرعيين أو أجهزة معينة.	3 أيام

المرحلة الخامسة: الأمان والتحسين، التجميع والاختبار النهائي (Sprint 5)

رمز المهمة	عنوان المهمة البرمجية للمطور	المدخلات والمخرجات المتوقعة (Deliverables)	الزمن المتوقع
BE-501	تحسين المداخل ومنع ثغرات حقن الأوامر (Command Injection Hardening)	تأمين الباك اند بالكامل ضد محاولات الهروب من الحاويات أو حقن أوامر خبيثة تؤثر على السيرفر الرئيسي للمنصة.	يومان
BE-502	إعداد نظام تتبع الأخطاء الشامل (Centralized Logging)	تفعيل موديول الـ Logging لتسجيل جميع الحركات داخل النظام وتتبع صحة الـ API Server و الـ Elastic Integration والـ Simulator.	يوم واحد
BE-503	إجراء اختبارات التكامل الشاملة (End-to-End API Testing)	تأكيد تواصل جميع المكونات البرمجية من كتابة القاعدة، واختبارها بالـ Sandbox وتحويل كودها بشكل ناجح 100%.	يومان

توجيه ختامي للمطور:

يجب مراعاة جعل استجابات الأخطاء (Error Responses) واضحة جداً وتفسيرية، خصوصاً في موديول رفع ملفات الـ YAML والـ Validation، لمساعدة مهندسي الواجهة الأمامية (Frontend) على إظهار رسائل خطأ دقيقة للمستخدم النهائي تصف بدقة موضع الخطأ النحوي داخل السكريبت.